End-to-End Data Management Pipeline for a Movie Recommendation System

Data Management for Machine Learning – Assignment 2

1. Abstract

This project implements an end-to-end data management and machine learning workflow for a movie recommendation system. It covers data ingestion, validation, preparation, feature engineering, transformation, model training, evaluation and structured Python workflow execution. A synthetic dataset containing 250 users, 120 movies and 15,000 ratings is used. Two recommendation approaches—popularity-based ranking and collaborative filtering using Truncated Singular Value Decomposition (TruncatedSVD)—are compared using ranking metrics. Rating-prediction error is also reported for collaborative filtering.

2. Dataset and Pipeline

The project uses three synthetic CSV files: users.csv (user information), movies.csv (movie metadata such as titles and genres) and ratings.csv (user–movie interactions and ratings). The data is synthetic and is not sourced from MovieLens.

The pipeline consists of eight stages: (1) data ingestion; (2) data validation; (3) data preparation; (4) feature engineering; (5) data transformation; (6) chronological 80:20 train-test split; (7) model training; and (8) model evaluation.

Validation checks missing values, duplicate records, invalid ratings, invalid timestamps and referential integrity between users, movies and ratings. Preparation handles missing or invalid records and removes duplicates. Feature engineering creates movie, user and interaction features. Transformation produces user-activity summaries, item-popularity statistics and a user–item interaction matrix.

Movie features include genre indicators and release decade. User features include age groups, occupation indicators and preferred-genre indicators. Interaction features include temporal fields such as rating year, month and day of week. The project stores engineered features in data/features/ and documents them in reports/feature_catalog.csv.

3. Models and Experimental Setup

The popularity-based model ranks movies using interaction information from the training data, including interaction counts and average ratings. It provides a non-personalized baseline.

The collaborative-filtering model applies TruncatedSVD to the user–item interaction matrix to learn latent representations and reconstruct predicted ratings. Personalized recommendations are generated from predicted scores while excluding movies already rated by a user in the training data.

The chronological split contains 12,000 training ratings and 3,000 test ratings. For ranking evaluation, ratings of 4 or above are treated as relevant and the top 10 recommendations are evaluated.

4. Results and Model Comparison

The ranking results show that the popularity-based approach has slightly higher Precision@10, Recall@10 and NDCG@10 in this experiment. Collaborative filtering achieves full catalogue coverage and slightly higher diversity, indicating that its recommendations span a broader set of movies.

Table 1. Recommendation ranking results

Metric	Popularity-based	Collaborative filtering
Precision@10	0.0504	0.0492
Recall@10	0.1495	0.1333
NDCG@10	0.1074	0.0929
Coverage	0.25	1.00
Diversity	0.7345	0.7501

Table 2. Rating-prediction results (collaborative filtering only)

Metric	Collaborative filtering
RMSE	2.8249
MAE	2.5711

RMSE and MAE are not applicable to the popularity-based ranking model in this implementation.

5. Experiment Tracking and Orchestration

MLflow was used during the project to track the popularity-based and collaborative-filtering experiments and their evaluation metrics. The local MLflow database is excluded from the public repository.

The workflow is launched through dags/run_pipeline.py. The script invokes the end-to-end pipeline, writes execution logs to reports/orchestration_run.log and saves a JSON execution summary to reports/orchestration_results.json. A successful run was recorded with 12,000 training records and 3,000 test records.

This implementation is a manually triggered, structured Python workflow. It is not an Airflow, Prefect or Dagster deployment, and periodic scheduling has not been configured. An external scheduler could be added for recurring execution.

6. Conclusion and Limitations

The project demonstrates an end-to-end data management workflow for a recommendation use case, including data quality checks, feature creation, model training, evaluation, experiment tracking and execution logging. The comparison shows a trade-off: the popularity baseline performs somewhat better on the reported top-10 relevance metrics, while collaborative filtering provides broader catalogue coverage and marginally higher diversity.

The experiment uses synthetic data, so the reported metrics demonstrate the implemented workflow rather than establish real-world recommendation quality. The user–item matrix represents unobserved interactions as zero, a simplifying modelling choice for TruncatedSVD. The workflow is manually triggered rather than deployed on a scheduler. Future work could evaluate on a real interaction dataset, explore alternative implicit-feedback methods and configure scheduled orchestration.